Extração e Preparação de Dados

Aula 14 – Engenharia de Software para Dados

Nivelamento (80/20) e Transformadores Customizados

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

luis.pinheiro@professores.ibemec.edu.br

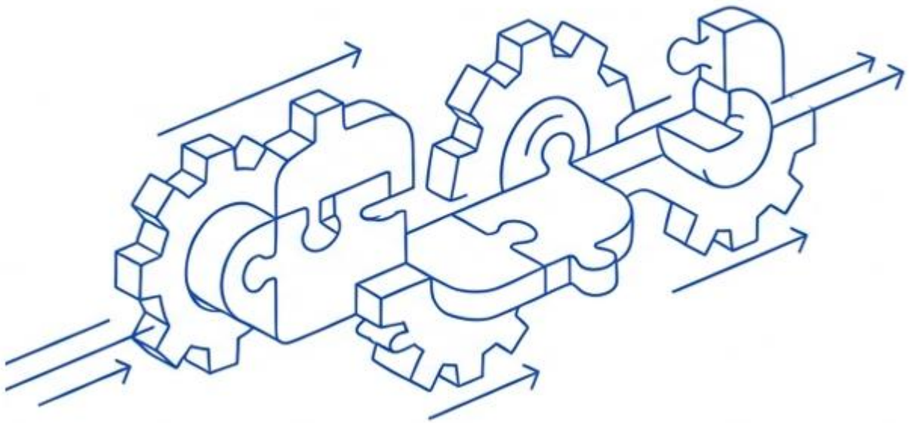
(96) 99907-5819

@l_aramis

Intro a Pipelines

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
# A estrutura base: Lista de Tuplas ('nome', Objeto())
meu_fluxo = Pipeline(steps=[
    ('preenchedor', SimpleImputer(strategy='median')),
    ('escalonador', StandardScaler())
])
```

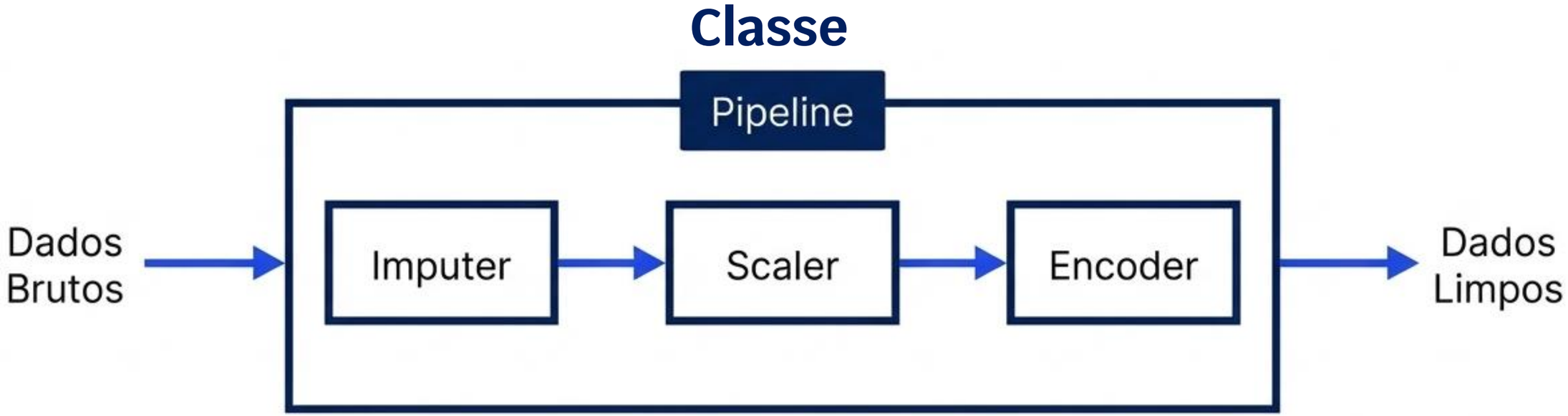


```
# A execução segura no mundo real
modelo_treinado = meu_fluxo.fit(X_treino, y_treino)

# O teste JAMAIS recalcula médias ou medianas
previsoes = meu_fluxo.transform(X_teste)
```

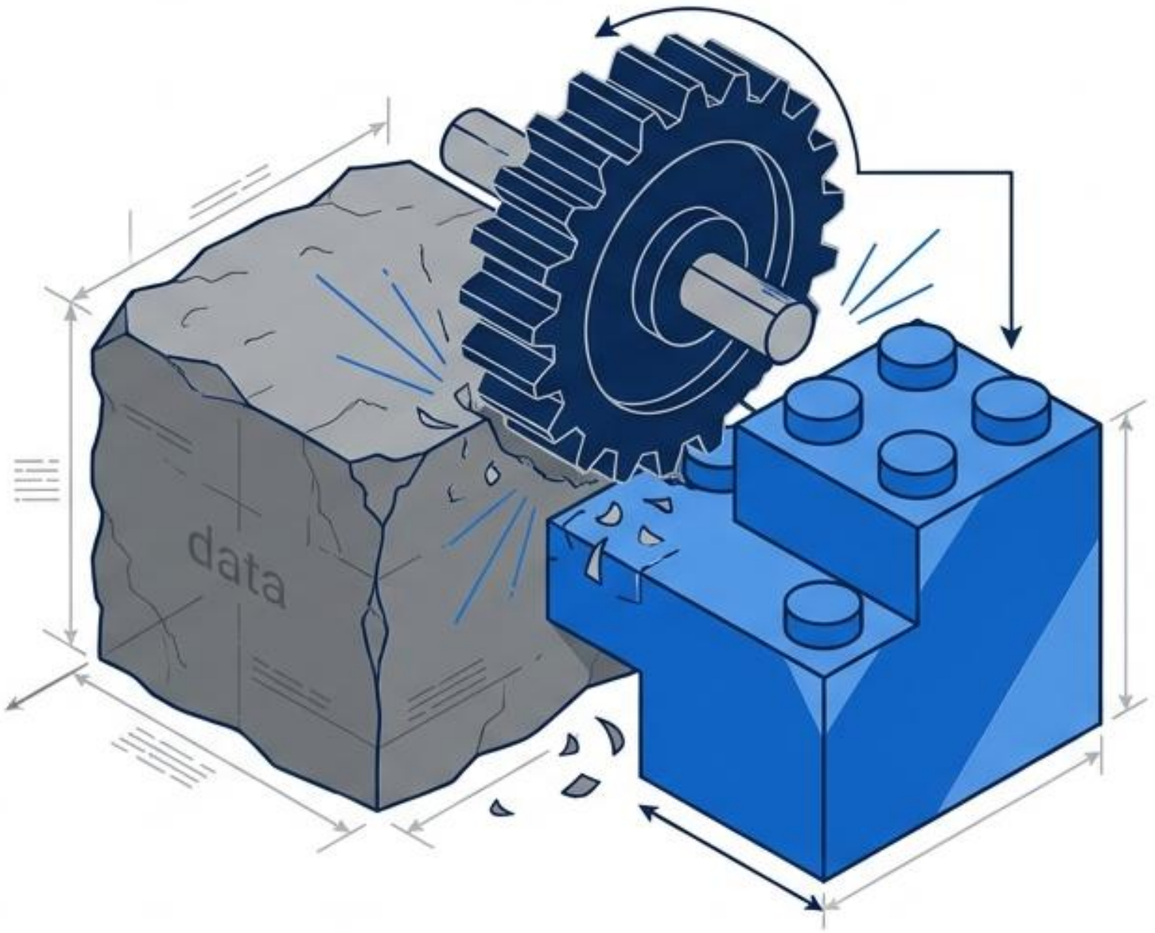
O Paradigma do Pipeline

- Introdução ao subpacote **sklearn.pipeline**.
- Encapsulamento de múltiplas transformações em um único objeto.
- Garantia de execução sequencial e proteção contra vazamentos.



Engenharia de Software para Dados

- Nivelamento Analítico: A Divisão 80/20 (Treino e Teste)
- Introdução à Programação Orientada a Objetos (POO)
- O Contrato Arquitetural do Scikit-Learn
- Construção de Transformadores Customizados



Nivelamento: O Aluno e a Prova

Treino (80%)

- **Fase de estudo.** O algoritmo descobre os padrões.
- **O Risco (Overfitting):** Decorar o gabarito destrói a capacidade de generalização.

Teste (20%)

- **Fase de validação.** O algoritmo é testado no escuro.
- **A Regra:** Dados de teste são invioláveis durante a preparação.

A Realidade Estatística: Treinar vs. Testar

- **Treinar (Fit):** É a etapa algébrica de otimização. A máquina varre os 80% dos dados para calcular e extrair parâmetros populacionais (como a média e o desvio padrão).
- **Testar (Transform):** É a etapa determinística. A máquina congela os parâmetros encontrados no treino e os aplica como uma equação estática sobre os 20% restantes.
- **O Risco Matemático:** Se calcularmos a média usando a base inteira, os dados do Teste vazam para o Treino (Data Leakage), invalidando a ciência.

```
# A execução segura no mundo real
modelo_treinado = meu_fluxo.fit(X_treino, y_treino)
```

StandardScaler()

1. FASE DE TREINO (DESCOBRINDO PARÂMETROS)

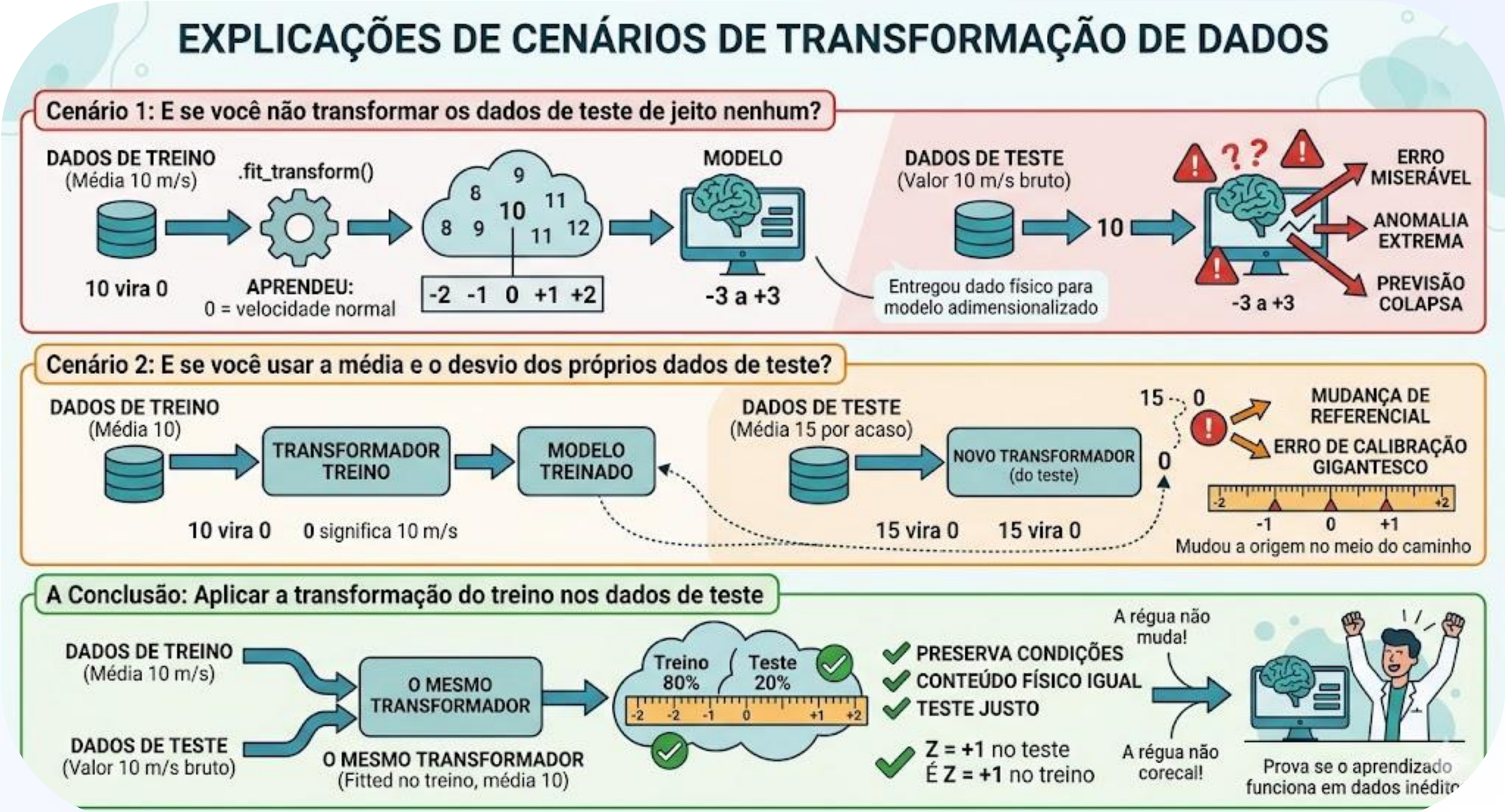
$$\mu_{treino} = \frac{1}{N} \sum_{i=1}^{N} x_i$$

2. FASE DE TESTE (APLICAÇÃO ESTÁTICA)

$$X_{escalado} = \frac{x_{novo} - \mu_{treino}}{\sigma_{treino}}$$

```
# O teste JAMAIS recalcula médias ou medianas
previsoes = meu_fluxo.transform(X_teste)
```

Por que treinar e testar?



A Anatomia do Vazamento (Exemplo Numérico)

Imagine uma base de idades. Treino (80%): **[20, 22, 24]**. Teste (20%): **[80]** (um novo cliente idoso fora do padrão).

Cenário Ideal (Parâmetro Congelado)

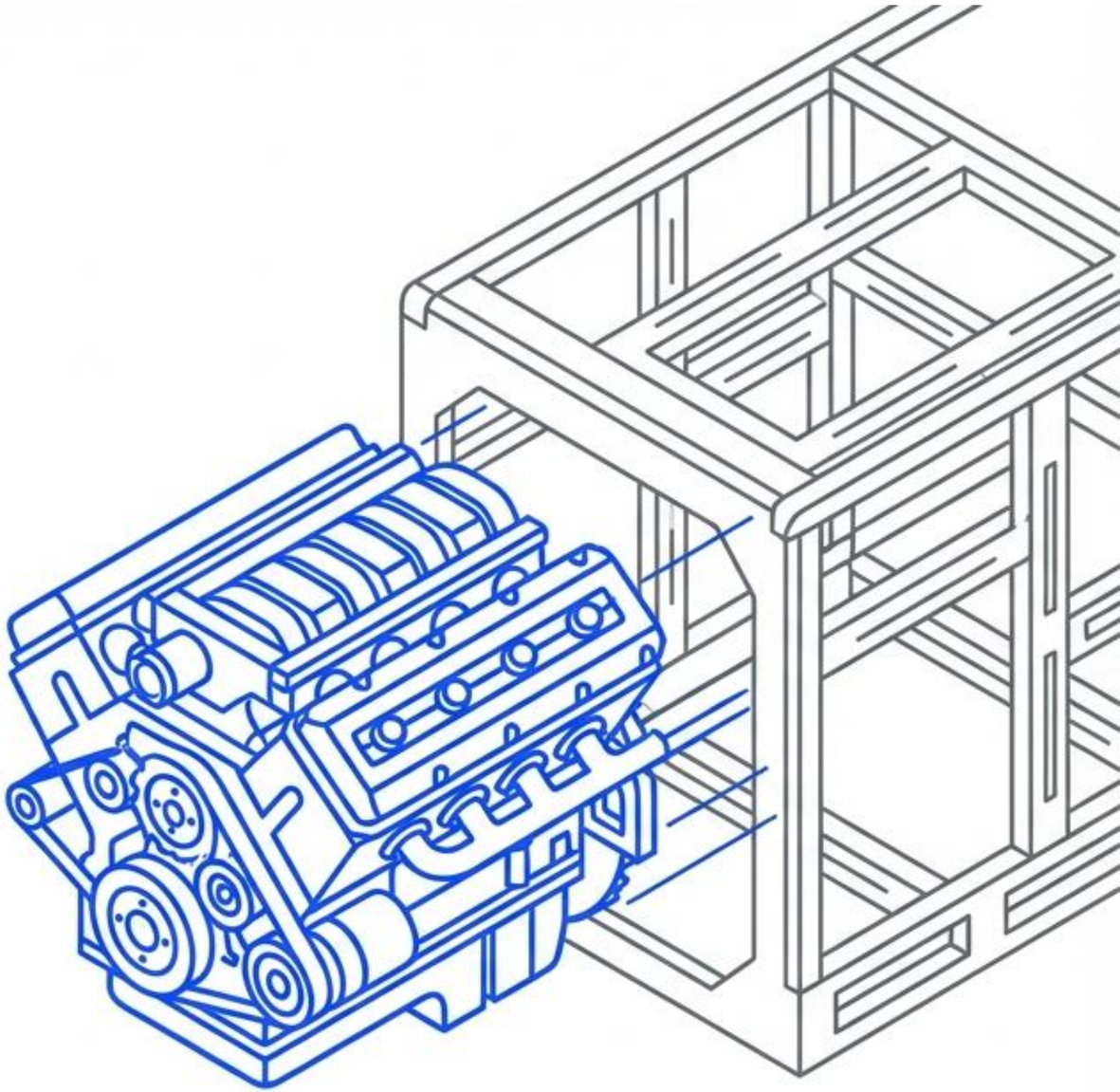
- **1. Fit no Treino:** A máquina calcula a Média = **22**
- **2. Transform no Teste:** A máquina avalia o novo cliente de 80 anos usando a régua fixa do 22.
- **Consequência:** A distância é enorme (+58). O modelo percebe que é um *outlier* legítimo e age corretamente. Ele **generalizou**.

Cenário Real (Data Leakage / Overfitting)

- **1. Fit em 100% da base:** A máquina "espia" tudo. Média = **36.5**
- **2. O Desastre:** A presença do 80 vazou para o treino. A régua "normal" subiu artificialmente.
- **Consequência:** Quando o 80 é escalado, ele parece menos extremo. O modelo "decorou" a base, distorceu os dados comuns e vai falhar em produção.

Transformadores Customizados na Prática

- Nivelamento Analítico e Modularização em Python
- Extração e Preparação de Dados (IBM8915)



O Artesão vs. O Engenheiro (A Fábrica de Lego)

O Trabalho Artesanal

Alterar dados manualmente no Pandas (**df['lucro']** = ...) não é sustentável em produção quando os dados chegam de madrugada em larga escala.

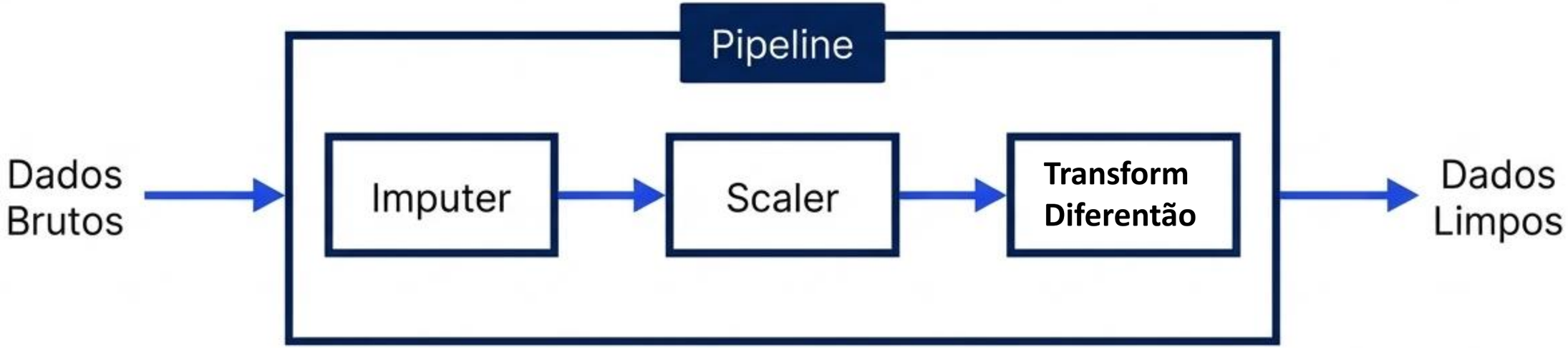
A Caixa de Lego

O **Pipeline** do Scikit-Learn nos dá "peças prontas" (**SimpleImputer**, **StandardScaler**) que se encaixam perfeitamente na linha de montagem.

O Desafio de Negócio

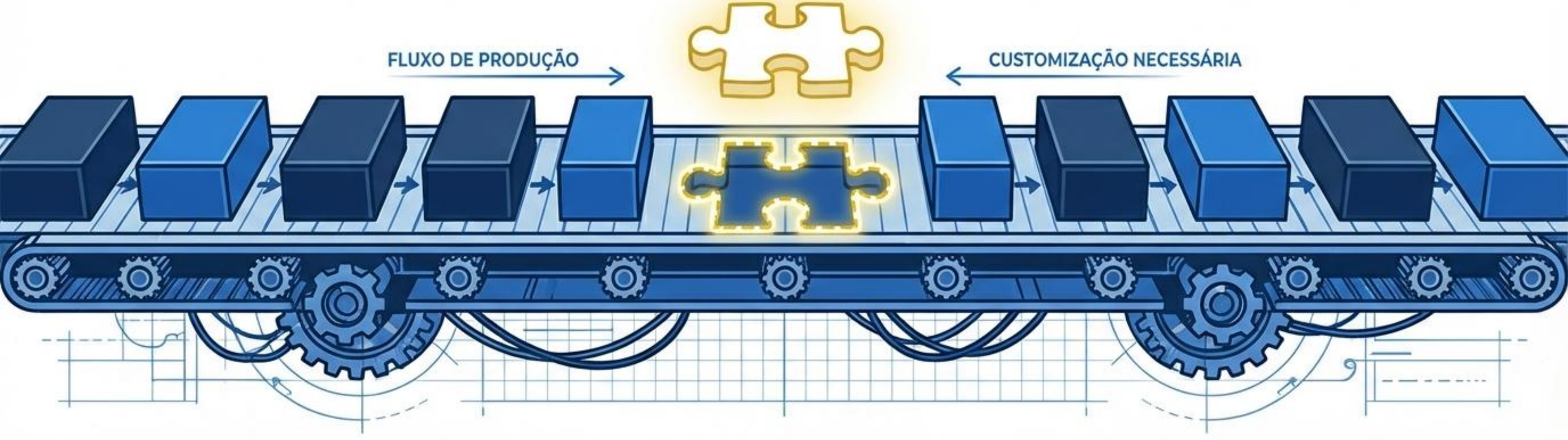
E quando a empresa precisa de uma peça que não existe na caixa oficial? Exemplo: Um **CalculadorDeTicketMedio** ou motor temporal exclusivo.

Pipeline


Pipeline
Dados Brutos
Imputer
Scaler
Transform Diferentão
Dados Limpos

A Necessidade de Peças Customizadas

- Limitações de bibliotecas padrão.
- **Regras** de **negócio** exclusivas da empresa.
- O conceito de **modularidade** (“Peças de Lego”).
- Transição do ambiente procedural para o arquitetural.



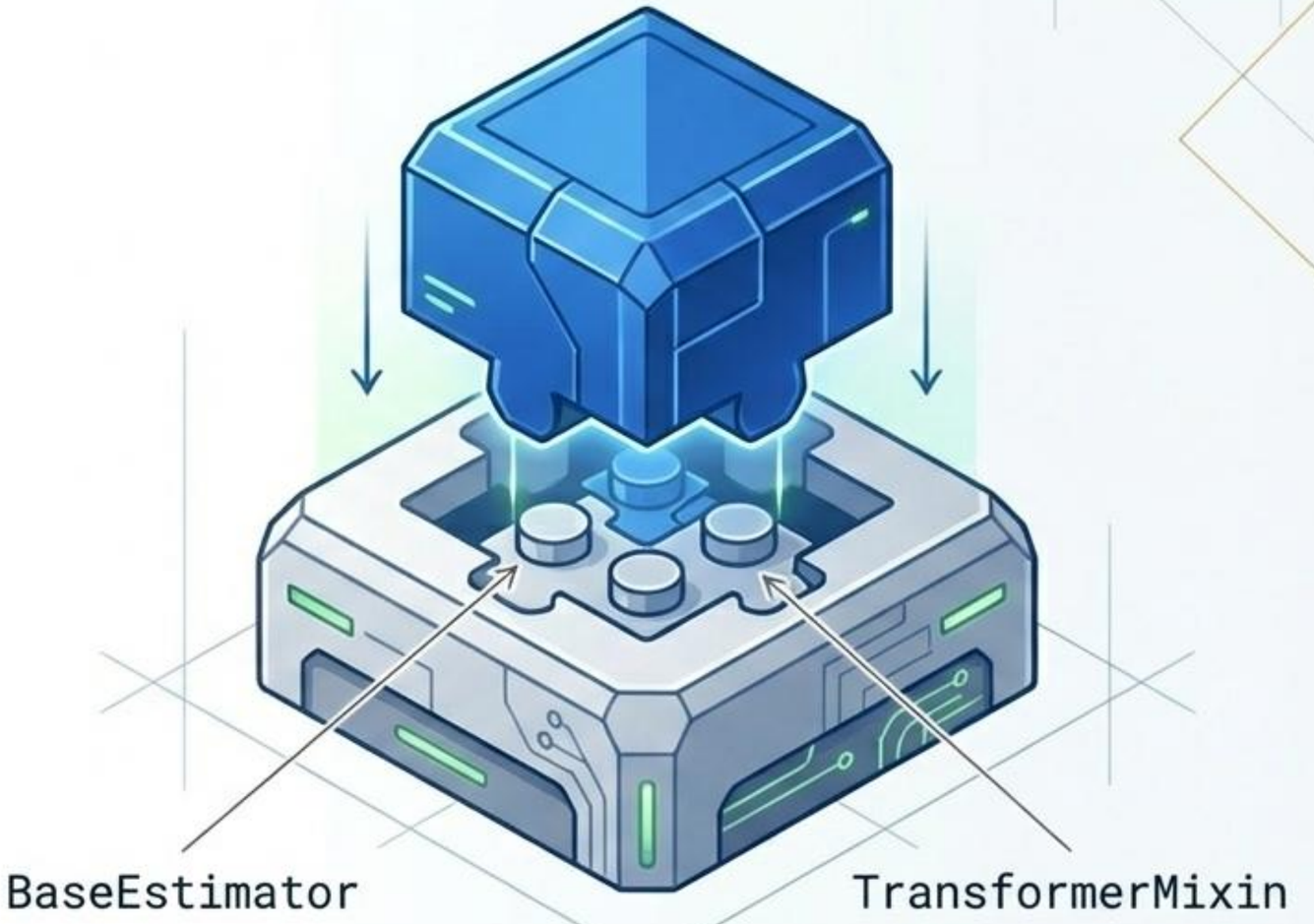
Fabricando a Máquina: POO em Python

```python
class MyMachine:
    def __init__(self, data):
        self.data = data
```

- A planta baixa ou molde de engenharia.
- O botão de ligar (método construtor da máquina).
- O “HD interno” que guarda as informações na memória.

O Contrato de Encaixe do Scikit-Learn

- Herança múltipla: **BaseEstimator** e **TransformerMixin**.
- Obrigatoriedade: Garantir compatibilidade com Pipelines contínuos.
- A função **fit**(self, X, y=None): Treinamento.
- A função **transform**(self, X): Execução.

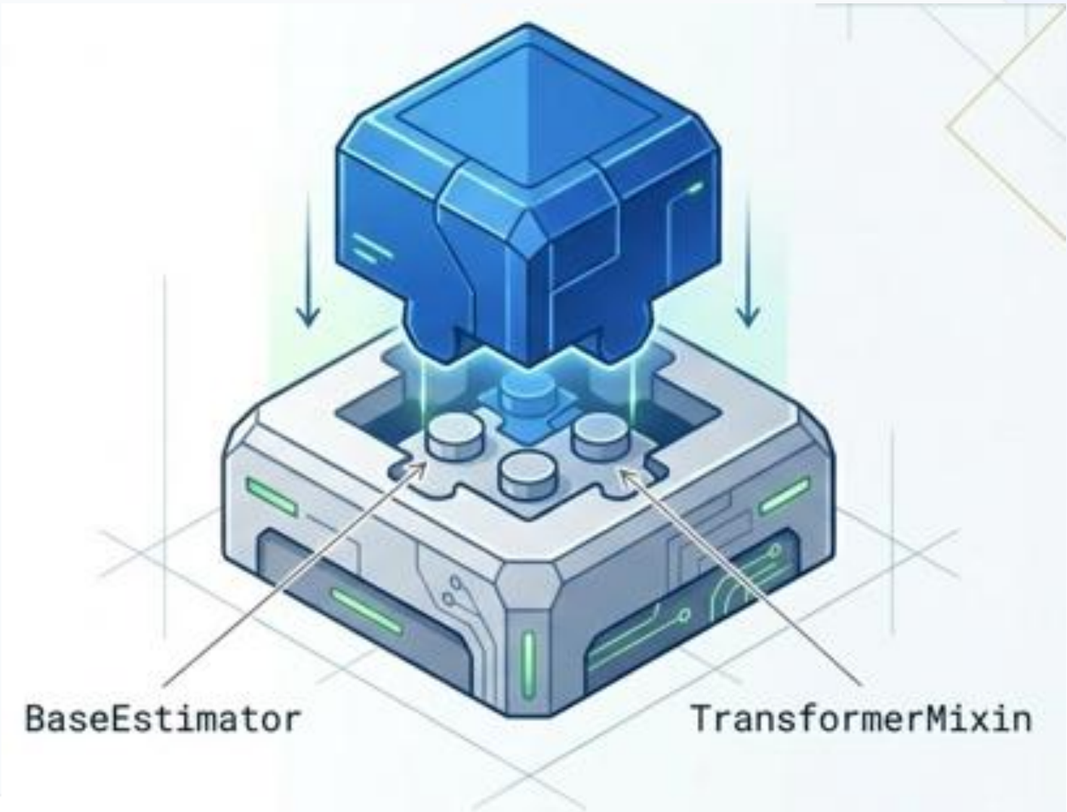


O Contrato de Encaixe do Scikit-Learn

```
from sklearn.base import BaseEstimator, TransformerMixin

class MeuTransformador(BaseEstimator, TransformerMixin):
```

- **A Herança de Classes:** Para a nossa peça funcionar na linha de montagem oficial, ela precisa herdar os moldes base.
- **BaseEstimator:** Garante a integração de hiperparâmetros (métodos get_params e set_params nativos).
- **TransformerMixin:** Nos dá o método **.fit_transform()** de graça, otimizando processamento.



Live Coding: O Esqueleto da Classe

- Passo 1: Imports do módulo sklearn.base.
- Passo 2: Estrutura da Classe e Método __init__.
- Passo 3: Implementação do fit (retornando self).
- Passo 4: Implementação do transform (retornando X).

```python
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin

class MeuTransformador(BaseEstimator, TransformerMixin):
    """
    Um transformador customizado de exemplo.
    """
    def __init__(self, parametro=None):
        """
        Inicialização do transformador com hiperparâmetros.
        """
        self.parametro = parametro
        print(f"Transformador inicializado com parametro={self.parametro}")

    def fit(self, X, y=None):
        """
        Método de treinamento. Deve retornar a própria instância (self).
        """
        print("Executando o método fit...")
        # Lógica de treinamento (calculo de médias, etc.) aqui.
        return self

    def transform(self, X):
        """
        Método de transformação. Deve retornar os dados transformados (X).
        """
        print("Executando o método transform...")
        X_transformado = X.copy()  # Exemplo de operação
        # Lógica de transformação aqui.
        return X_transformado
```

```python
class MultiplicadorCustomizado(BaseEstimator, TransformerMixin):
    # O __init__ é onde você define os hiperparâmetros (opcional)
    def __init__(self, fator=2):
        self.fator = fator

    # O fit é obrigatório.
    # Se o seu transformador não precisa memorizar nada (como médias),
    # ele deve apenas retornar 'self'.
    def fit(self, X, y=None):
        return self

    # O transform é onde a modificação dos dados acontece.
    def transform(self, X, y=None):
        # Multiplica os dados pelo fator escolhido
        X_modificado = X * self.fator
        return X_modificado
```

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# Instanciando dentro da lista de passos (steps) do Pipeline
meu_pipeline = Pipeline(steps=[
    ('multiplicador', MultiplicadorCustomizado(fator=10)), # Seu transformador cu
    ('padronizador', StandardScaler()),                    # Transformador nativo
    ('modelo', KNeighborsClassifier(n_neighbors=3))        # O modelo final
])

# Usando o pipeline completo
# O pipeline vai chamar o .fit_transform() do seu Multiplicador,
# passar o resultado para o .fit_transform() do StandardScaler,
# e por fim passar os dados para o .fit() do modelo KNN.
meu_pipeline.fit(X_train, y_train)

# Na hora de testar, o pipeline vai chamar apenas o .transform() de todos os pass
previsoes = meu_pipeline.predict(X_test)
```

Check de Aprendizado (A Regra de Ouro)

- O Contrato exige a criação de duas funções principais:
  - `fit(self, X, y=None)`
  - `transform(self, X)`

Qual dessas duas funções é a Fase de Estudo (80%) e qual é a Prova Final (20%)?

Hands-on: Fabricando o Próprio Lego (PBL)

1. Desenvolver

Preencher a classe **ConversorTemporal** no Jupyter garantindo o pleno funcionamento estatístico dos métodos fit e transform.

2. Testar

Rodar um teste unitário simples em uma célula isolada para garantir que a classe processa o DataFrame sem retornar exceções de tipo ou formato.

3. Exportar (Modularizar)

Abrir o VS Code, criar o script **custom_transformers.py**, colar o código testado e salvar como biblioteca oficial da equipe.

Injetando a Lógica de Negócio

Encapsulamento da Regra: A lógica de 'Fim de Semana' (Aula 10) atua como o motor interno.

O Papel do .transform(): Recebe os dados brutos, aplica a transformação cirúrgica e retorna a matriz atualizada.

Integração Contínua: O dado flui pela linha de montagem sem necessidade de variáveis temporárias no Jupyter.

The Shell

```
class ConversorTemporal:
    def transform(self, X):
```

The Engine

```
X['Fim_de_Semana'] =
X['Data'].dt.dayofweek >= 5
```

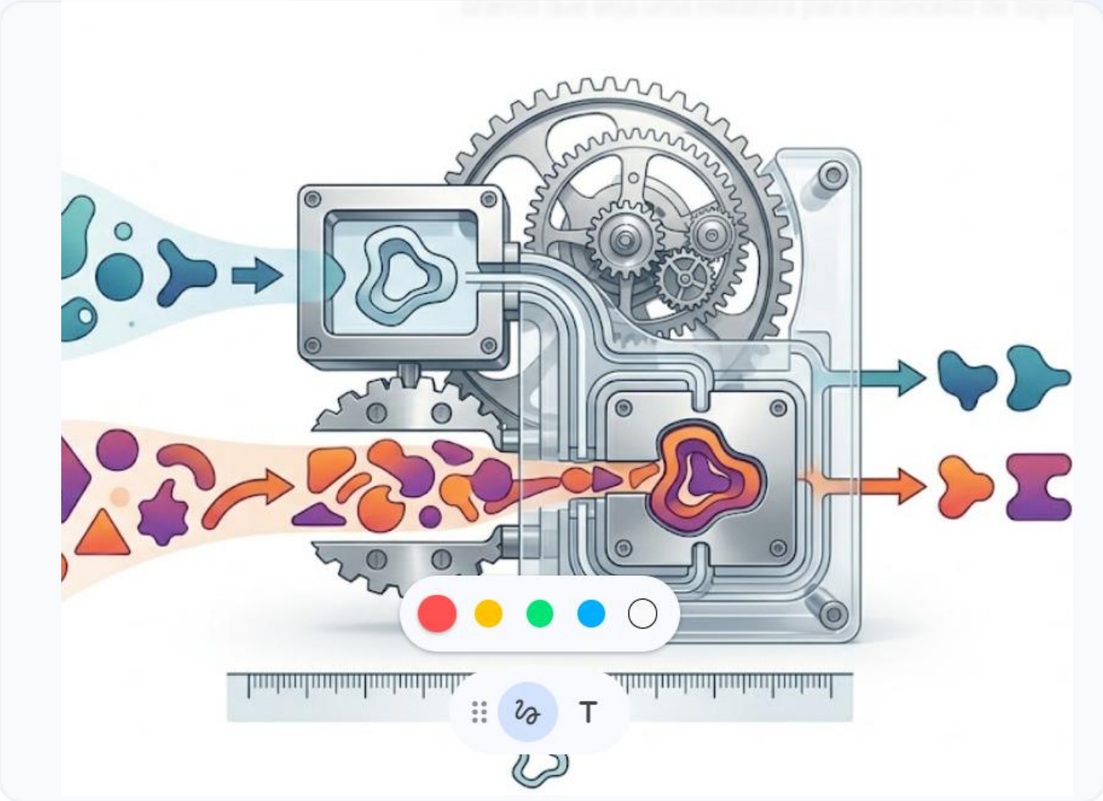
O Coração da Máquina (Lógica no Transform)

```python
def transform(self, X):
    # Criando uma cópia para segurança
    X_modificado = X.copy()

    # Injetando a regra de negócio da Aula 10
    X_modificado['Data'] =
pd.to_datetime(X_modificado['Data'])
    X_modificado['Is_Weekend'] =
(X_modificado['Data'].dt.dayofweek >= 5).astype(int)

    return X_modificado
```

- **A Missão:** Trazer códigos Pandas soltos para dentro do contrato rigoroso do Scikit-Learn.
- **O Isolamento:** Usar **X.copy()** garante a integridade do dado original na memória.
- **A Automação:** A linha de montagem agora infere a flag preditiva temporal automaticamente.

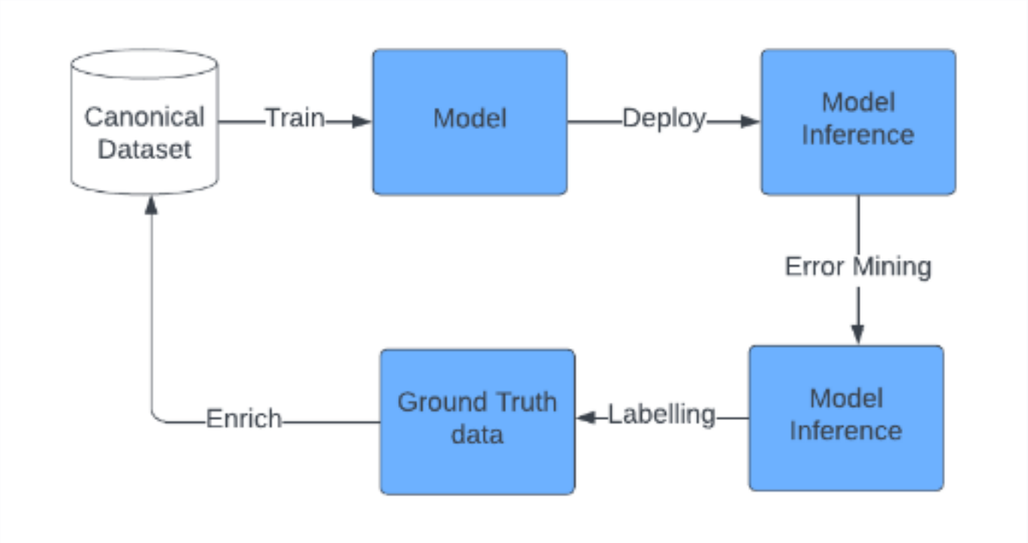


Fechando a Linha de Montagem

```python
from custom_transformers import ConversorTemporal
from sklearn.pipeline import Pipeline

# Integrando a nossa peça com as peças oficiais
fluxo_final = Pipeline(steps=[
    ('tempo', ConversorTemporal()),
    ('scaler', StandardScaler()),
    ('modelo', RandomForestClassifier())
])
```

- **A Importação:** O Jupyter atua apenas como o "painel de controle" orquestrando arquivos externos modulares.
- **O Casamento Perfeito:** A classe customizada opera lado a lado com os algoritmos nativos.
- **O Teste Real:** Ao acionar **.fit(X_treino)**, a matemática e o nivelamento analítico são processados sem vazamentos.



Fabricando e Exportando o Componente

1 Desenvolver

Preencher a classe **ConversorTemporal** no Jupyter, garantindo o funcionamento integrado do **fit** e **transform**.

2 Testar

Executar validação rigorosa em uma célula isolada para comprovar que a classe não retorna erros de sintaxe ou vazamento.

3 Exportar

Migrar para o **VS Code**, criar o arquivo **custom_transformers.py**, colar a estrutura de classe e salvar para produção.

Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec